

Secure User Management System: Security Assessment and Hardening Report

Cybersecurity Developershub Internship | Professional Submission Report | June 2026

Executive Summary

This report documents the assessment and hardening of a mock Express User Management System with signup, login, and profile workflows. The final application implements input validation, output encoding, bcrypt password hashing, JWT authentication, Helmet headers, rate limiting, Winston logging, and automated tests.

Submission Requirements Mapping

Requirement	Delivered Item	Status
Report document	Professional_Cybersecurity_Project_Report.docx / PDF	Complete
GitHub repository	cybersecurity-user-management-submission project folder	Ready to upload
Video explanation	Video script and run steps prepared	Must be recorded
Checklist	docs/security_checklist.md	Complete

Application Scope

- Application: Secure User Management System.
- Stack: Node.js, Express, HTML/CSS, local JSON mock storage.
- Pages tested: /signup, /login, /profile.
- Local target: http://localhost:3000.

Assessment Methodology

- Reviewed signup, login, and profile workflows.
- Tested XSS with `<script>alert('XSS');</script>`.
- Tested SQL-injection-style login input `admin' OR '1'='1`.
- Inspected password storage, authentication, headers, logs, tests, and npm audit.

Findings and Remediation

Finding	Risk	Evidence	Remediation
XSS through profile text	High	Payload entered: <code><script>alert('XSS');</script></code>	HTML-escape profile output before rendering.
SQL-injection-style login input	High	Payload entered: <code>admin' OR '1'='1</code>	Validate email input and avoid unsafe query concatenation.
Weak password storage	High	Password storage inspected after signup.	Hash passwords with bcrypt and never return hashes in API responses.
Unauthenticated profile access	Medium	Profile/API profile requested without login token.	Require JWT authentication for profile routes.

Finding	Risk	Evidence	Remediation
Missing browser security headers	Medium	HTTP response headers inspected.	Enable Helmet and Content Security Policy.
Limited audit trail	Medium	Authentication events reviewed.	Add Winston request, login, signup, failed-login, and token logs.

Security Controls Implemented

- validator input validation and sanitization.
- HTML output encoding for profile data.
- bcrypt password hashing.
- JWT authentication for protected profile access.
- Helmet HTTP headers and CSP.
- express-rate-limit on authentication endpoints.
- Winston security logging.
- Automated security regression tests.

Verification Results

Verification	Result
Invalid email rejected	Passed
Password stored as bcrypt hash	Passed
SQL-injection-style login payload rejected	Passed
Profile API requires JWT	Passed
XSS payload escaped on profile page	Passed
Helmet headers present and x-powered-by removed	Passed
npm audit	0 vulnerabilities found

Evidence Screenshots

Figure 1. Home page of the secure user-management system.

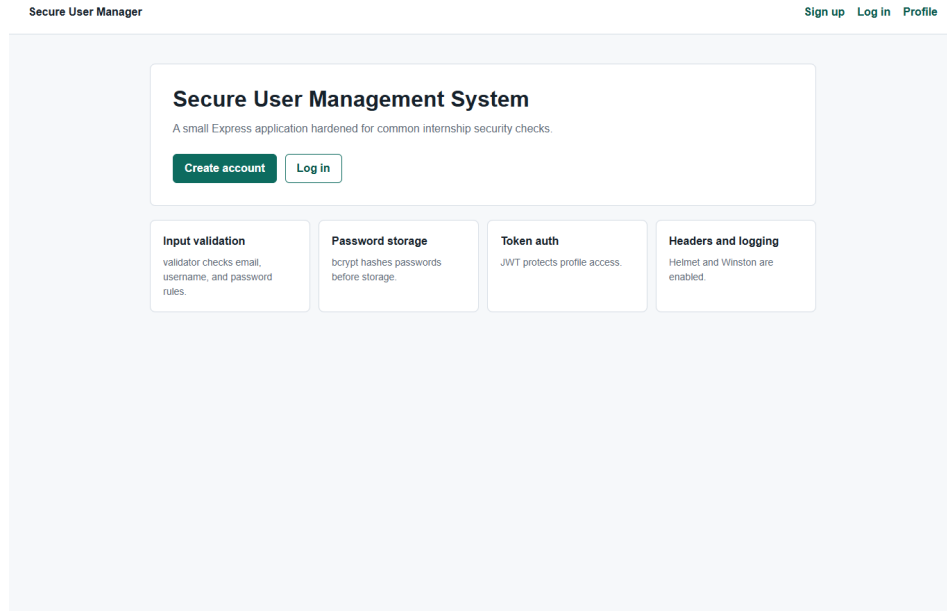


Figure 2. Signup page with the XSS test payload entered in the full-name field.

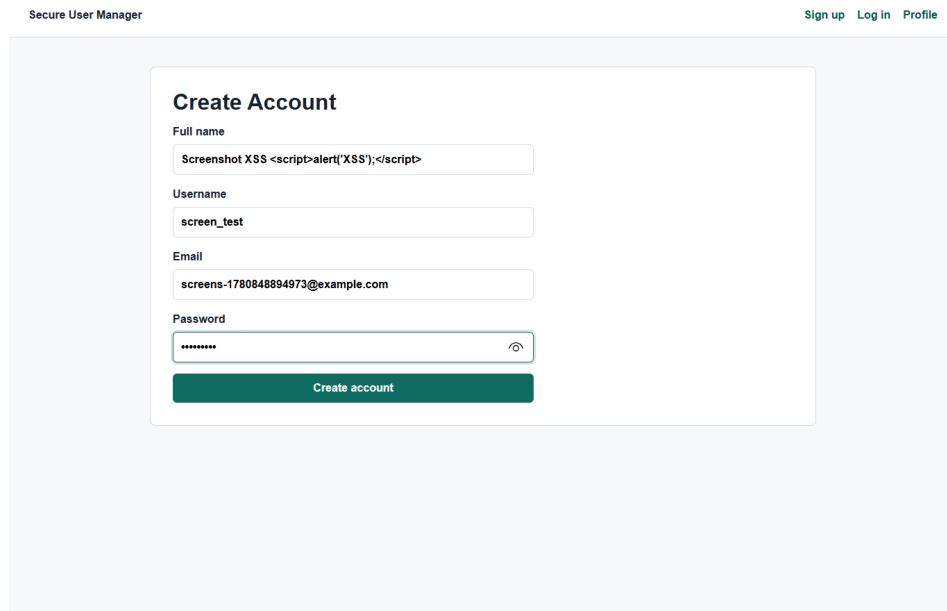


Figure 3. Login page after successful account creation.

Secure User Manager Sign up Log in Profile

Log In

Account created. Log in with your credentials.

Email

Password

Log in

Figure 4. Profile page showing the XSS payload as visible escaped text, not executable script.

Secure User Manager Sign up Log in Profile

Profile

Full name	Screenshot XSS <script>alert('XSS');</script>
Username	screen_test
Email	screens-1780848894973@example.com
Created	2026-06-07T16:14:57.442Z

Log out

Figure 5. Automated tests and npm audit results.

```
Security Test Output

> npm test

PASS  signup rejects invalid email input
PASS  passwords are stored as bcrypt hashes, not plaintext
PASS  SQL injection style login payload is rejected
PASS  JWT is required for profile API access and works after login
PASS  profile page escapes reflected XSS payloads
PASS  Helmet security headers are present

Test summary:
  tests: 6
  pass: 6
  fail: 0

> npm run security:audit

found 0 vulnerabilities
```

Figure 6. GitHub repository contents prepared for upload.

```
GitHub Repository Contents

> repository folder structure
cybersecurity-user-management-submission/
src/
  server.js      # Express app, validation, auth, headers, logging
  userStore.js   # mock local user storage
public/
  styles.css     # UI styling
test/
  security.test.js # security regression tests
docs/
  week1_vulnerability_assessment.md
  security_checklist.md
  zap_nmap_test_plan.md
  final_report.md
  video_explanation_script.md
  test_results.md
package.json
package-lock.json
README.md
SECURITY.md
.env.example
.gitignore

Do not upload node_modules, .env, generated logs, or real user data.
```

Figure 7. Local run commands for a second PC.

```
Local Run Commands

> run on any PC with Node.js installed
PS> cd cybersecurity-user-management-submission
PS> npm install
PS> copy .env.example .env
PS> npm start

Expected output:
Secure User Manager running at http://localhost:3000

Open in browser:
http://localhost:3000

In another PowerShell window:
PS> npm test
PS> npm run security:audit
```

Figure 8. Helmet security headers and hidden Express fingerprint evidence.

```
HTTP Security Headers Evidence

> response security headers from http://localhost:3000
content-security-policy: script-src 'self';style-src 'self';object-src 'none';frame-
  ancestors 'none';default-src 'self';base-uri 'self';font-src 'self' https: data:;form-
  action 'self';img-src 'self' data:;script-src-attr 'none';upgrade-insecure-requests
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
strict-transport-security: max-age=31536000; includeSubDomains
referrer-policy: no-referrer
x-powered-by: NOT PRESENT (Express fingerprint hidden)
OK: Helmet security headers are active.
```

Figure 9. Bcrypt password-storage evidence.

```
Password Storage Evidence

> stored user record check
email: screens-1780848894973@example.com
username: screen_test
passwordHash: $2b$12$yaORCAIQE1K74Z.GbQPYU...
plaintext password stored: NO
bcrypt hash prefix found: YES

Finding: passwords are not stored as plaintext.
Control: bcrypt hashes and salts passwords before storage.
```

Figure 10. SQL-injection-style login payload rejection evidence.

```
SQL Injection Test Evidence

> SQL-injection-style login test
Payload used in login:
admin' OR '1'='1

Expected secure result:
- Input rejected as invalid email or invalid credentials
- No authentication bypass
- No profile access granted

Automated test result:
PASS SQL injection style login payload is rejected
```

Limitations and Production Recommendations

- Replace the mock JSON store with a production database using parameterized queries.
- Enable HTTPS in production.

- Store JWT_SECRET in a secure environment variable.
- Add CSRF protection if browser cookie flows expand.
- Send logs to centralized monitoring in production.
- Run OWASP ZAP/Nmap where those tools are installed and authorized.

Conclusion

The final submission provides secure code, findings, fixes, tests, screenshots, checklist, and GitHub-ready files. The remaining actions are uploading the repository and recording the required video explanation.